

Hands-on 4: Difference between JPA, Hibernate and Spring Data JPA

1. JPA (Java Persistence API)

JPA (Java Persistence API) is a Java specification (JSR 338) for persisting, reading, and managing data from Java objects. It defines standard APIs for Object Relational Mapping (ORM) but does not provide an implementation. Hibernate is one of the most popular implementations of JPA.

2. Hibernate

Hibernate is an ORM framework that implements the JPA specification. It manages sessions, transactions, object mapping, and automatically generates SQL queries.

Hibernate Code Snippet

```
Session session = factory.openSession();
Transaction tx = session.beginTransaction();

session.save(employee);

tx.commit();
session.close();
```

3. Spring Data JPA

Spring Data JPA is an abstraction layer built on top of JPA. It does not implement JPA itself; instead, it uses JPA providers such as Hibernate. It reduces boilerplate code, automatically manages transactions, and provides ready-made repository methods such as `save()`, `findById()`, and `delete()`.

Spring Data JPA Code Snippet

```
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {
}

@Service
public class EmployeeService {

    @Autowired
    private EmployeeRepository employeeRepository;

    @Transactional
    public void addEmployee(Employee employee) {
        employeeRepository.save(employee);
    }
}
```

Comparison Table

Feature	JPA	Hibernate	Spring Data JPA
Type	Specification	ORM Framework	Abstraction Layer
Implementation	No	Yes	No (Uses Hibernate)
Purpose	Defines persistence standard	Implements JPA	Simplifies JPA usage
Boilerplate Code	High	Medium	Very Low
Transaction Handling	Manual	Manual / Automatic	Automatic (@Transactional)
Ease of Use	Medium	Medium	High

Conclusion

JPA is a specification, Hibernate is its implementation, and Spring Data JPA is a higher-level abstraction that makes database operations easier by reducing boilerplate code while internally using Hibernate.